



Security Assessment Report

Static Application Security Testing & Code Assessment

PROJECT

Personal Project

SCAN NAME

444444

UPLOADED FILE

Vulnerable.java

ASSESSMENT TYPE

Initial Assessment

LANGUAGE

java

ASSESSMENT DATE

4 October 2026

PREPARED BY

SecureCode Auditor Platform

CONFIDENTIAL

Document Control

Document Title	Security Assessment Report
Application	SecureCode Auditor
Project	Personal Project
Scan Name	444444
Uploaded File	Vulnerable.java
Assessment Type	INITIAL
Scan ID	6ac20bc05b3659d746f2505c
Programming Language	java
Scanner	SecureCode Auditor Platform
Assessment Date	4/10/2026, 1:48:08 pm
Report Generated	5/10/2026, 9:46:57 pm
Report Version	1.0
Classification	CONFIDENTIAL

Table of Contents

- 1. Executive Summary
- 2. Assessment Overview
- 3. Assessment Methodology
- 4. Scope
- 5. Risk Summary
- 6. Findings Summary
- 7. Detailed Findings
 - 7.1 Critical Findings
 - 7.2 High Findings
 - 7.3 Medium Findings
- 8. Remediation & Recommendations
- 9. Conclusion
- 10. References
- 11. Appendix

1. Executive Summary

This security assessment was automatically performed using the SecureCode Auditor platform to identify, classify, and provide remediation guidance for security weaknesses within the assessed source code.



Overall Security Risk: **CRITICAL**

The assessed codebase contains critical severity findings that pose an immediate and severe threat to application security. Immediate prioritized remediation is mandatory to prevent potential compromise.

2. Assessment Overview

Project	Personal Project
Scan Name	444444
Uploaded File	Vulnerable.java
Language	java
Scanner	SecureCode Auditor Platform
Scan Type	INITIAL
Assessment Date	4/10/2026, 1:48:08 pm

3. Assessment Methodology

The security assessment follows a rigorous lifecycle leveraging both static analysis and AI-assisted contextual validation:

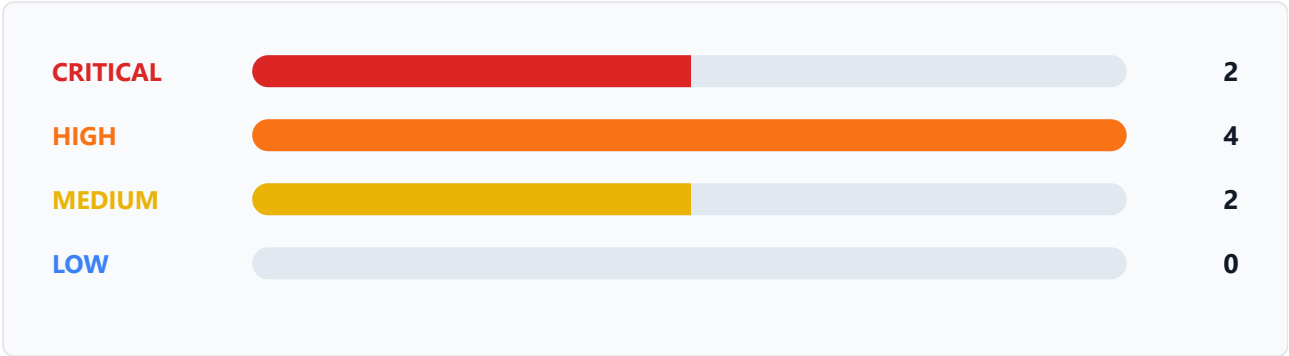
- 1. **Static Analysis:** The source code is scanned using industry-standard static application security testing (SAST) engines to identify vulnerable patterns.
- 2. **Vulnerability Identification:** Findings are mapped to standard frameworks (CWE, OWASP).
- 3. **AI-Assisted Analysis:** Validated findings are analyzed by an advanced AI model to determine context, root cause, and precise secure code remediation.
- 4. **Manual Security Review:** Human security analysts review critical findings to validate True/False positives and assess business risk.
- 5. **Reporting:** A comprehensive assessment report is compiled detailing all evidence and actionable remediation steps.

4. Scope

The scope of this assessment includes the source code provided to the SecureCode Auditor platform during the initiation of the scan.

Project Scope	Personal Project
Target Language	java
Environment	Static Source Code

5. Risk Summary



6. Findings Summary

ID	Finding	Severity	Status	CWE	Review
F-001	SQL String Concatenation	CRITICAL	OPEN	CWE-89	N/A
F-002	Missing Parameterized Queries	CRITICAL	OPEN	CWE-89	N/A
F-003	java-sql-injection	HIGH	OPEN	N/A	N/A
F-004	java-command-injection	HIGH	OPEN	N/A	N/A
F-005	java-unsafe-deserialization	HIGH	OPEN	N/A	N/A
F-006	Weak Encryption	HIGH	CONFIRMED	CWE-327	TR
F-007	java-hardcoded-secret	MEDIUM	OPEN	N/A	N/A
F-008	java-weak-cryptography	MEDIUM	OPEN	N/A	N/A

7. Detailed Findings

F-001

SQL String Concatenation CRITICAL

Status	OPEN	Scanner	N/A
Rule ID	SCA-SQL-001	File	Vulnerable.java
CWE	CWE-89	Line	25
OWASP	A03 Injection		

Description

Detect SQL queries constructed using untrusted input.

Vulnerable Code

```
"SELECT * FROM users WHERE username = '" +
```

Security Impact

Confidence: HIGH

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

REFERENCES

- [R1]** **CWE-89 — CWE-89**
External Reference
<https://cwe.mitre.org/data/definitions/89.html>
- [R2]** **OWASP Category — A03 Injection**
External Reference
<https://owasp.org/>

F-002

Missing Parameterized Queries **CRITICAL**

Status	OPEN	Scanner	N/A
Rule ID	SCA-SQL-004	File	Vulnerable.java
CWE	CWE-89	Line	28
OWASP	A03 Injection		

Description

Detect database queries that don't use prepared statements or parameterized APIs.

Vulnerable Code

```
statement.executeQuery(query);
```

Security Impact

Confidence: HIGH

AI-ASSISTED SECURITY ANALYSIS

Summary

Direct execution of dynamic SQL queries using standard Statement instead of PreparedStatement causes SQL Injection (CWE-89).

Why It Is Vulnerable

The application invokes `statement.executeQuery(query)` using a standard JDBC `java.sql.Statement` instance. When the SQL query string passed to `executeQuery` is constructed using string concatenation or formatting with untrusted user input, malicious input can alter the syntax and structure of the executed SQL command. This crosses the boundary between application control flow and the database execution engine without proper input-output separation.

Root Cause

Constructing dynamic SQL statements and executing them through `java.sql.Statement` rather than binding variables using parameterized queries (`java.sql.PreparedStatement`).

Recommendation

Refactor the database access logic to use `java.sql.PreparedStatement` with placeholder characters (?) instead of direct string manipulation, and bind inputs using appropriate type-specific setter methods (e.g., `setString`, `setInt`).

Secure Code Example

```
String sql = "SELECT id, name, role FROM users WHERE username = ? AND status = ?";
try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
    pstmt.setString(1, username);
    pstmt.setString(2, status);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            // process result set
        }
    }
}
```

REFERENCES

- [R1]** **CWE-89 — CWE-89**
External Reference
<https://cwe.mitre.org/data/definitions/89.html>
- [R3]** **owasp.org/Top10/A03_2021-Injection/**
OWASP
https://owasp.org/Top10/A03_2021-Injection/
- [R4]** **cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html**
OWASP
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [R2]** **OWASP Category — A03 Injection**
External Reference
<https://owasp.org/>

F-003

java-sql-injection **HIGH**

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	Vulnerable.java
CWE	N/A	Line	24
OWASP	N/A		

Description

Potential SQL injection detected.

Security Impact

Impact: HIGH

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

F-004

java-command-injection

HIGH

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	Vulnerable.java
CWE	N/A	Line	33
OWASP	N/A		

Description

Using Runtime.exec() is dangerous if input is not sanitized.

Security Impact

Impact: HIGH

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

F-005

java-unsafe-deserialization HIGH

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	Vulnerable.java
CWE	N/A	Line	47
OWASP	N/A		

Description

Unsafe deserialization of objects can lead to remote code execution.

Security Impact

Impact: HIGH

AI-ASSISTED SECURITY ANALYSIS

Summary

The application deserializes untrusted data using Java's native ObjectInputStream, which can lead to remote code execution (RCE) if an attacker provides a malicious serialized object.

Why It Is Vulnerable

Java's native deserialization mechanism (ObjectInputStream) reconstructs objects from a byte stream. If the stream contains a maliciously crafted serialized object (e.g., a gadget chain), the deserialization process can trigger arbitrary code execution. The vulnerability occurs because the application does not validate or restrict the types of objects that can be deserialized, allowing untrusted data to be processed without safeguards.

Root Cause

The application uses Java's native deserialization (ObjectInputStream) on untrusted input without implementing a whitelist of allowed classes or using a safer serialization format.

Recommendation

1. Avoid deserializing untrusted data using Java's native serialization. 2. If native deserialization is required, implement a whitelist of allowed classes using `ObjectInputFilter` (Java 9+) or a custom `ObjectInputStream`. 3. Prefer safer serialization formats like JSON or Protocol Buffers for data exchange.

Secure Code Example

```
import java.io.*;
import java.util.*;

public class SafeDeserialization {
    private static final Set<String> ALLOWED_CLASSES = new HashSet<>(Arrays.asList(
        "com.example.MySafeClass",
        "java.lang.String"
    ));

    public static Object safeDeserialize(byte[] data) throws IOException, ClassNotFoundException
    {
        try (ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(data)) {
            @Override
            protected Class<?> resolveClass(ObjectStreamClass desc) throws IOException,
            ClassNotFoundException {
                if (!ALLOWED_CLASSES.contains(desc.getName())) {
                    throw new InvalidClassException("Unauthorized deserialization attempt",
                    desc.getName());
                }
                return super.resolveClass(desc);
            }
        }) {
            return ois.readObject();
        }
    }
}
```

REFERENCES

[R5] OWASP Deserialization Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html

OWASP

OWASP Deserialization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html

[R6] Deserialization of Untrusted Data: <https://cwe.mitre.org/data/definitions/502.html>

MITRE CWE

CWE-502: Deserialization of Untrusted Data: <https://cwe.mitre.org/data/definitions/502.html>

[R7] Java ObjectInputFilter Documentation:

<https://docs.oracle.com/javase/9/docs/api/java/io/ObjectInputFilter.html>

Oracle Documentation

Java ObjectInputFilter Documentation: <https://docs.oracle.com/javase/9/docs/api/java/io/ObjectInputFilter.html>

F-006

Weak Encryption HIGH

Status	CONFIRMED	Scanner	N/A
Rule ID	SCA-CRYPTO-005	File	Vulnerable.java
CWE	CWE-327	Line	41
OWASP	A02 Cryptographic Failures		

Description

Detect DES, 3DES, RC4, ECB and other inappropriate cryptographic configurations.

Vulnerable Code

```
// 5. Unsafe deserialization
```

Security Impact

Confidence: HIGH

AI-ASSISTED SECURITY ANALYSIS

Summary

The application uses weak or obsolete cryptographic algorithms or insecure cipher modes (such as DES, 3DES, RC4, or ECB mode), which fail to provide adequate confidentiality and integrity for sensitive data.

Why It Is Vulnerable

Cryptographic algorithms such as DES, 3DES, and RC4 have known mathematical weaknesses, small key sizes, or vulnerability to practical collision and plaintext recovery attacks (e.g., Sweet32 for 64-bit block ciphers like 3DES). Additionally, using Electronic Codebook (ECB) mode with any block cipher leaks structural patterns because identical plaintext blocks encrypt to identical ciphertext blocks. Without an authenticated encryption mode (like GCM), encrypted data is also susceptible to bit-flipping and padding oracle attacks. In Java, initiating `Cipher.getInstance("DES")` or `Cipher.getInstance("AES/ECB/PKCS5Padding")` introduces these vulnerabilities across data protection boundaries.

Root Cause

Insecure cryptographic configuration resulting from the selection of broken algorithms (DES, RC4, 3DES) or unauthenticated/unrandomized cipher modes (ECB).

Recommendation

Replace legacy algorithms and modes with strong, modern authenticated encryption algorithms, specifically AES in Galois/Counter Mode (AES/GCM/NoPadding) using a 128-bit or 256-bit key and a cryptographically secure random 12-byte initialization vector (IV) generated per encryption operation.

Secure Code Example

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import java.security.SecureRandom;

public class CryptoUtil {
    private static final int GCM_TAG_LENGTH = 128;
    private static final int IV_LENGTH_BYTE = 12;

    public static byte[] encrypt(byte[] plaintext, SecretKey key) throws Exception {
        byte[] iv = new byte[IV_LENGTH_BYTE];
        SecureRandom.getInstanceStrong().nextBytes(iv);

        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        GCMParameterSpec spec = new GCMParameterSpec(GCM_TAG_LENGTH, iv);
        cipher.init(Cipher.ENCRYPT_MODE, key, spec);
        byte[] ciphertext = cipher.doFinal(plaintext);

        byte[] combined = new byte[iv.length + ciphertext.length];
        System.arraycopy(iv, 0, combined, 0, iv.length);
        System.arraycopy(ciphertext, 0, combined, iv.length, ciphertext.length);
        return combined;
    }
}
```

REFERENCES

[R8] **cwe.mitre.org/data/definitions/327.html**
MITRE CWE
<https://cwe.mitre.org/data/definitions/327.html>

[R9] **owasp.org/Top10/A02_2021-Cryptographic_Failures/**
OWASP
https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

[R10] **cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html**
OWASP
https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

[R11] **csrc.nist.gov/publications/detail/sp/800-38d/final**
NIST
<https://csrc.nist.gov/publications/detail/sp/800-38d/final>

[R2] **OWASP Category — A03 Injection**
External Reference
<https://owasp.org/>

MANUAL SECURITY REVIEW

Decision	TRUE_POSITIVE	Reviewer Risk	High
Comments	need to fix at high priority		

F-007

java-hardcoded-secret MEDIUM

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	Vulnerable.java
CWE	N/A	Line	9
OWASP	N/A		

Description

Hardcoded secret detected. This can lead to sensitive information exposure.

Security Impact

Impact: HIGH

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

F-008

java-weak-cryptography MEDIUM

Status	OPEN	Scanner	N/A
Rule ID	N/A	File	Vulnerable.java
CWE	N/A	Line	37
OWASP	N/A		

Description

MD5 is a weak cryptographic algorithm.

Security Impact

Impact: HIGH

AI-ASSISTED SECURITY ANALYSIS

AI analysis was not generated for this finding.

8. Remediation & Recommendations

ID	Severity	Recommended Action
F-001	CRITICAL	Use parameterized queries instead of string concatenation.
F-002	CRITICAL	Refactor the database access logic to use <code>java.sql.PreparedStatement</code> with placeholder characters ('?') instead of direct string manipulation, and bind inputs using appropriate type-specific setter methods (e.g., <code>setString</code> , <code>setInt</code>).
F-003	HIGH	Use <code>PreparedStatement</code> instead of concatenating strings.
F-004	HIGH	Use <code>ProcessBuilder</code> and avoid passing raw strings directly.
F-005	HIGH	1. Avoid deserializing untrusted data using Java's native serialization. 2. If native deserialization is required, implement a whitelist of allowed classes using <code>ObjectInputFilter</code> (Java 9+) or a custom <code>ObjectInputStream</code> . 3. Prefer safer serialization formats like JSON or Protocol Buffers for data exchange.
F-006	HIGH	Replace legacy algorithms and modes with strong, modern authenticated encryption algorithms, specifically AES in Galois/Counter Mode (AES/GCM/NoPadding) using a 128-bit or 256-bit key and a cryptographically secure random 12-byte initialization vector (IV) generated per encryption operation.
F-007	MEDIUM	Use environment variables or a secret manager.
F-008	MEDIUM	Use SHA-256 or stronger algorithms.

9. Conclusion

The assessment of **Personal Project** resulted in the identification of **8** vulnerabilities, resulting in an overall risk rating of **CRITICAL**.

The assessed codebase contains critical severity findings that pose an immediate and severe threat to application security. Immediate prioritized remediation is mandatory to prevent potential compromise.

It is strongly recommended that the development team reviews the detailed findings in Section 7 and applies the remediation steps outlined in Section 8. Critical and High severity findings should be prioritized in the immediate sprint to minimize organizational risk. Upon completion of remediation efforts, a retest scan should be performed to validate the integrity of the implemented fixes.

10. References

Reference Traceability

Finding	References
F-001	[R1], [R2]
F-002	[R1], [R3], [R4], [R2]
F-003	No external references available.
F-004	No external references available.
F-005	[R5], [R6], [R7]
F-006	[R8], [R9], [R10], [R11], [R2]
F-007	No external references available.
F-008	No external references available.

Master Reference Registry

[R1] CWE-89 — CWE-89

Source: External Reference
<https://cwe.mitre.org/data/definitions/89.html>
Used by: **F-001, F-002**

[R2] OWASP Category — A03 Injection

Source: External Reference
<https://owasp.org/>
Used by: **F-001, F-002, F-006**

[R3] owasp.org/Top10/A03_2021-Injection/

Source: OWASP
https://owasp.org/Top10/A03_2021-Injection/
Used by: **F-002**

[R4] cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html**Source:** OWASPhttps://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.htmlUsed by: **F-002****[R5] OWASP Deserialization Cheat Sheet:****https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html****Source:** OWASPOWASP Deserialization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.htmlUsed by: **F-005****[R6] Deserialization of Untrusted Data: <https://cwe.mitre.org/data/definitions/502.html>****Source:** MITRE CWECWE-502: Deserialization of Untrusted Data: <https://cwe.mitre.org/data/definitions/502.html>Used by: **F-005****[R7] Java ObjectInputStream Documentation:****<https://docs.oracle.com/javase/9/docs/api/java/io/ObjectInputStream.html>****Source:** Oracle DocumentationJava ObjectInputStream Documentation: <https://docs.oracle.com/javase/9/docs/api/java/io/ObjectInputStream.html>Used by: **F-005****[R8] cwe.mitre.org/data/definitions/327.html****Source:** MITRE CWE<https://cwe.mitre.org/data/definitions/327.html>Used by: **F-006****[R9] owasp.org/Top10/A02_2021-Cryptographic_Failures/****Source:** OWASPhttps://owasp.org/Top10/A02_2021-Cryptographic_Failures/Used by: **F-006****[R10] cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html****Source:** OWASPhttps://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.htmlUsed by: **F-006**

[R11] csrc.nist.gov/publications/detail/sp/800-38d/final

Source: NIST

<https://csrc.nist.gov/publications/detail/sp/800-38d/final>

Used by: **F-006**

11. Appendix

A. Scan Metadata

Scan ID	6ac20bc05b3659d746f2505c
Generated On	5/10/2026, 9:46:57 pm
Platform Engine	SecureCode Auditor Core

B. AI Model Information

The following AI models were utilized during the analysis of findings in this report:

- gemini-3.8-flash
- qwen/qwen3.8-27b

Note: AI analysis is strictly limited to static technical context and never introduces fabricated telemetry or false references.